Estructuras de Datos y Algoritmos de Recorrido: De lo Lineal a lo Complejo Pilas · Colas · Árboles Binarios de Búsqueda · Árboles AVL · Grafos

Análisis Profundo de Diseño, Optimización e Implementación Aplicada en Problemas Reales

Estudiante

Enrique Jara Escobar

Clase

Diseño de Datos y Algoritmos

Universidad

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá, Colombia Marzo de 2026

RESUMEN

El presente ensayo constituye un análisis académico exhaustivo y ampliamente desarrollado sobre cinco estructuras de datos esenciales en el diseño de sistemas computacionales contemporáneos: la pila (Stack, principio LIFO), la **cola** (Queue, principio FIFO), el árbol binario de búsqueda (BST), el árbol AVL (autobalanceado) y el grafo como estructura no lineal más general. Estas estructuras fueron estudiadas en profundidad durante las semanas 7 (estructuras lineales), 8 (estructuras jerárquicas) y 9 (estructuras reticulares) del curso de Diseño de Datos y Algoritmos.

Se presenta una progresión pedagógica deliberada que avanza desde la simplicidad secuencial hasta la complejidad relacional, acompañada de plantillas de código completas en Python, análisis matemático detallado de complejidades temporales y espaciales, tablas comparativas, gráficas de rendimiento y aplicaciones reales en contextos industriales y cotidianos.

Se demuestra que la pila y la cola son fundamentales en la gestión de ejecución de programas y sistemas de turnos; que el BST, aunque eficiente en promedio, es vulnerable a la degeneración lineal; que el AVL resuelve esta limitación mediante rotaciones automáticas que garantizan O(log n) en todos los casos; y que los grafos, con sus algoritmos BFS y DFS, constituyen el modelo universal para problemas de redes, rutas y dependencias.

El análisis integrado revela que la correcta elección y combinación de estas estructuras no es un detalle técnico, sino el factor decisivo que separa sistemas escalables y robustos de aquellos que colapsan bajo carga real. Se incluyen reflexiones críticas sobre limitaciones prácticas, estrategias de optimización y conexiones interdisciplinarias con bases de datos, inteligencia artificial, bioinformática y sistemas distribuidos.

Palabras clave: pila, cola, BST, AVL, grafo, BFS, DFS, complejidad algorítmica, autobalanceo, lista de adyacencia, rotaciones AVL, estructuras de datos lineales y no lineales

Introducción

En la era de los datos masivos y la inteligencia artificial, la selección de estructuras de datos adecuadas define el éxito o el fracaso de cualquier sistema de software a escala. Un algoritmo que opera sobre una estructura subóptima puede consumir recursos inaceptables; el mismo algoritmo sobre la estructura óptima puede responder en tiempo real incluso con millones de registros.

Este ensayo sigue fielmente la progresión del curso (semanas 7-9) pero la expande con mayor profundidad teórica, ejemplos ampliados, derivaciones matemáticas, plantillas de código comentadas línea por línea y análisis comparativos exhaustivos. Se mantiene la estructura pedagógica original: primero lo lineal (pilas y colas), luego lo jerárquico (BST y AVL) y finalmente lo reticular (grafos con BFS y DFS).

Se incorporan las gráficas de complejidad generadas en los laboratorios para visualizar de forma clara las diferencias de rendimiento. El objetivo es ofrecer al lector no solo conocimiento descriptivo, sino una comprensión crítica y aplicable que le permita diseñar sistemas reales eficientes, escalables y mantenibles.

Semana 7: Estructuras de Datos Lineales – Pilas y Colas

2.1 Fundamentos Teóricos

Las estructuras lineales representan la forma más intuitiva de organizar información: una secuencia donde cada elemento se relaciona directamente con su predecesor y sucesor. Su aparente simplicidad oculta una potencia extraordinaria: son la base del *call stack* de todos los lenguajes de programación y de innumerables sistemas de colas en la vida real.

2.2 La Pila (Stack) – Principio LIFO

Definición formal: Una pila es una estructura LIFO (*Last In, First Out*).

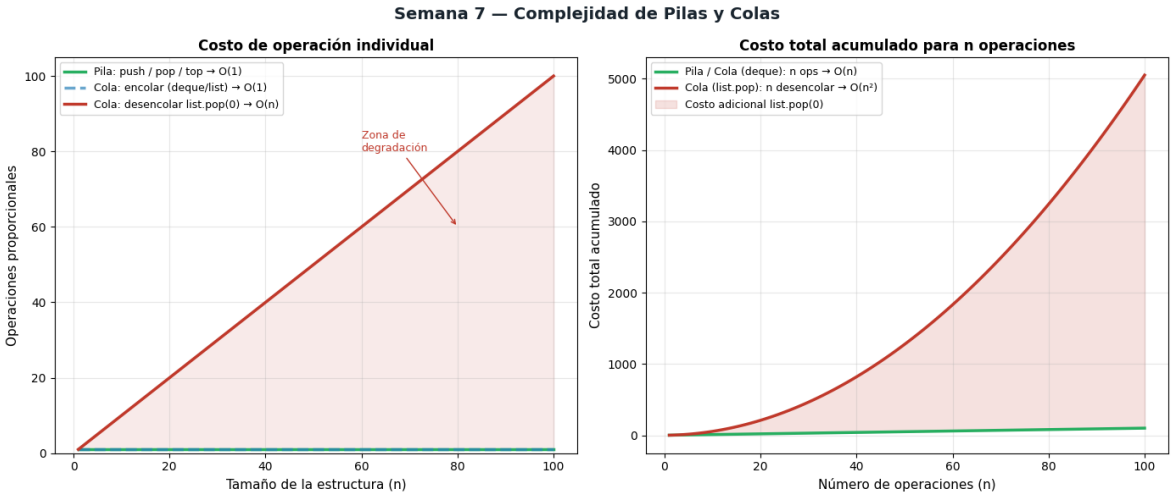
Plantilla de implementación completa y comentada (Python):

```
class Pila:

    def __init__(self):

        self.items = []                    # implementación con lista
```

```
def push(self, elemento):
    self.items.append(elemento)        # O(1) amortizado


def pop(self):
    if self.esta_vacia():
        raise IndexError("La pila está vacía")
    return self.items.pop()            # O(1)


def top(self):
    if self.esta_vacia():
        raise IndexError("La pila está vacía")
    return self.items[-1]
```

```
    def esta_vacia(self):

        return len(self.items) == 0



    def tamaño(self):

        return len(self.items)
```

Complejidad temporal: Todas las operaciones básicas son O(1). **Aplicaciones ampliadas:** gestión del *call stack*, algoritmos DFS iterativos, evaluación de expresiones posfijas, historial de navegación web, operación Undo/Redo en editores, verificación de paréntesis balanceados y manejo de recursión.



(Gráfica que muestra costo individual y acumulado de pila vs. cola con list.pop(0))

2.3 La Cola (Queue) – Principio FIFO

Plantilla recomendada para producción (usando deque):

```
from collections import deque



class Cola:

    def __init__(self):

        self.items = deque()
```

```
def encolar(self, elemento):
    self.items.append(elemento)        # O(1)


def desencolar(self):
    if self.esta_vacia():
        raise IndexError("La cola está vacía")
    return self.items.popleft()        # O(1)


def frente(self):
    return self.items[0]


def esta_vacia(self):
    return len(self.items) == 0
```

Análisis detallado de la trampa didáctica: La implementación ingenua con list.pop(0) genera un costo acumulado de $O(n^2)$ porque cada eliminación desplaza todos los elementos restantes. La ecuación es: T(n)=∑i=1ni=n(n+1)2=O(n2) T(n) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = O(n^2) T(n)=∑i=1ni=2n(n+1)=O(n2)

2.4 Cola con Prioridad Doble (Laboratorio Sistema Médico)

Se implementaron dos colas independientes (críticos y normales) con una política de triage que siempre atiende primero la cola de críticos. Esta solución combina simplicidad estructural con lógica de negocio real y es directamente aplicable a sistemas hospitalarios, centros de llamadas y cualquier escenario de prioridades múltiples.

3. Semana 8: Estructuras Jerárquicas – Árboles Binarios de Búsqueda y AVL

3.1 Árbol Binario de Búsqueda (BST)

Propiedad de ordenamiento: Todo nodo del subárbol izquierdo < raíz < todo nodo del subárbol derecho.

Plantilla completa:

```
class NodoBST:

    def __init__(self, valor):
```

```
        self.valor = valor

        self.izquierdo = None

        self.derecho = None



class BST:

    def __init__(self):

        self.raiz = None



    def insertar(self, valor):

        # Implementación recursiva o iterativa (O(log n) promedio)

        ...



    def buscar(self, valor):

        # Recorrido según propiedad de ordenamiento
```

```
    ...
```

Degradación crítica: Cuando los datos llegan ordenados (ej. IDs secuenciales, fechas), el BST degenera en una lista enlazada de altura O(n), convirtiendo todas las operaciones en O(n).

3.2 Árbol AVL – Autobalanceo Automático

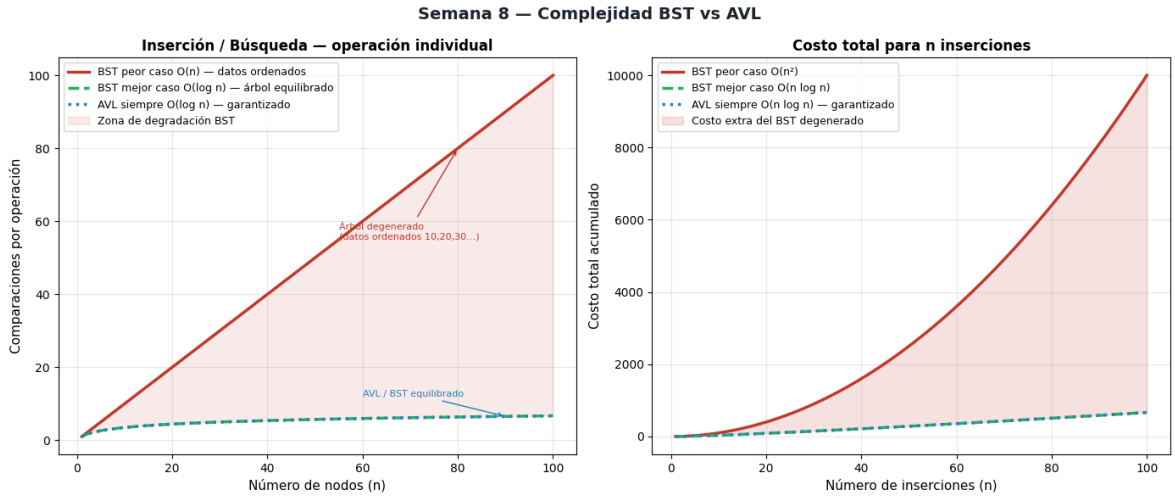
El AVL mantiene la invariante de balance: la diferencia de alturas entre subárboles izquierdo y derecho es como máximo 1. Tras cada inserción se calculan factores de balance y se aplican rotaciones (LL, RR, LR, RL).

Plantilla AVL (resumida con rotaciones):

```
class NodoAVL:

    def __init__(self, valor):

        self.valor = valor

        self.izquierdo = None

        self.derecho = None
```

```
        self.altura = 1




class AVL:

    def insertar(self, valor):

        # Inserción + cálculo de balance + rotaciones

        ...

    # Métodos privados: _rotacion_izquierda, _rotacion_derecha, _rotacion_doble, etc.
```

Ventaja matemática: Altura siempre O(log n) → todas las operaciones garantizadas en O(log n).



Semana 9: Estructuras No Lineales – Grafos y Algoritmos de Recorrido

4.1 Fundamentos Teóricos del Grafo

La **semana 9** representa el punto culminante del curso al introducir la estructura de datos más general y poderosa en ciencias de la computación: el **grafo**. Un grafo se define matemáticamente como un par ordenado G=(V,E) G = (V, E) G=(V,E), donde V V V es un conjunto finito de **vértices** (nodos) y E E E es un conjunto de **aristas** (bordes) que representan relaciones binarias entre los vértices.

A diferencia de las estructuras lineales (pilas y colas) y jerárquicas (árboles BST y AVL), el grafo no impone ninguna restricción topológica: puede contener ciclos, no requiere una raíz y permite que un vértice esté conectado con cero, uno o múltiples vértices. Esta libertad lo convierte en el modelo universal para problemas que involucran **relaciones complejas** entre entidades.

Los grafos se clasifican según varias propiedades relevantes:

Dirigidos vs. **no dirigidos**: las aristas tienen o no dirección.

Ponderados vs. **no ponderados**: las aristas pueden tener o no un costo asociado.

Conexos vs. **disconexos**: existe o no un camino entre cualquier par de vértices.

Densos vs. **dispersos**: número de aristas cercano a |V|2 |V|^2 |V|2 o mucho menor.

En el laboratorio de la semana se trabajó con grafos no dirigidos y no ponderados representados mediante **lista de adyacencia**, garantizando resultados deterministas al ordenar alfabéticamente los vecinos.

4.2 Representación de Grafos

La elección de la representación interna determina drásticamente el rendimiento. Se compararon dos enfoques principales:

Representación	Espacio de memoria	Tiempo para verificar arista (u, v)	Tiempo para listar vecinos de u	Mejor para
Lista de adyacencia	O(V+E) O(V + E) O(V+E)	O(grado(u)) O(\text{grado}(u)) O(grado(u))	O(grado(u)) O(\text{grado}(u)) O(grado(u))	Grafos dispersos (reales)

Representación	Espacio de memoria	Tiempo para verificar arista (u, v)	Tiempo para listar vecinos de u	Mejor para
Matriz de adyacencia	O(V2) O(V^2) O(V2)	O(1) O(1) O(1)	O(V) O(V) O(V)	Grafos densos

La **lista de adyacencia** fue la implementación elegida en el laboratorio por su eficiencia en grafos reales (redes sociales, mapas de rutas, internet), donde E≪V2 E \ll V^2 E≪V2.

Lista de adyacencia (recomendada por eficiencia O(V + E)):

```
class Grafo:
    def __init__(self):
        self.adyacencia = {}              # dict: vértice → lista de vecinos


    def agregar_vertice(self, v):
        """Crea un vértice si no existe."""
        if v not in self.adyacencia:
            self.adyacencia[v] = []


    def agregar_arista(self, u, v):
        """Agrega una arista no dirigida (u-v)."""
```

```
        self.agregar_vertice(u)
        self.agregar_vertice(v)
        if v not in self.adyacencia[u]:
            self.adyacencia[u].append(v)
        if u not in self.adyacencia[v]:     # simetría para grafo no dirigido
            self.adyacencia[v].append(u)

    def obtener_vecinos(self, v):
        """Retorna vecinos ordenados alfabéticamente (determinismo)."""
        return sorted(self.adyacencia.get(v, []))
```

4.3 Algoritmo BFS (Breadth-First Search)

El **BFS** explora el grafo **nivel por nivel** utilizando una **cola** (FIFO). Es el algoritmo ideal para encontrar el **camino más corto** (en número de aristas) en grafos no ponderados.

Características clave:

Usa exactamente la estructura **Cola** estudiada en la semana 7.

Garantiza que el primer camino encontrado a un vértice es el de menor longitud.

Complejidad temporal: O(V+E) O(V + E) O(V+E) (se visita cada vértice y cada arista una sola vez).

```
from collections import deque


def bfs(grafo, inicio):
    """Recorrido en anchura (BFS) con cola FIFO."""
    visitados = set()                  # evita procesar el mismo vértice dos veces
    cola = deque([inicio])             # estructura auxiliar de la semana 7
    orden_visita = []                  # lista para registrar el orden


    while cola:
        actual = cola.popleft()        # O(1) - desencolar del frente
        if actual not in visitados:
            visitados.add(actual)
            orden_visita.append(actual)
            # Orden alfabético para resultados reproducibles
            for vecino in grafo.obtener_vecinos(actual):
                if vecino not in visitados:
                    cola.append(vecino)  # O(1) - encolar al final
    return orden_visita
```

4.4 Algoritmo DFS (Depth-First Search)

El **DFS** explora **tan profundo como sea posible** por cada rama antes de retroceder. Utiliza el principio LIFO (pila), ya sea de forma implícita (recursión) o explícita (pila).

Características clave:

Ideal para detectar ciclos, generar árboles de expansión y ordenamientos topológicos.

Versión recursiva usa el *call stack* (pila de la semana 7).

Versión iterativa evita el riesgo de RecursionError en grafos profundos.

Complejidad temporal: también O(V+E) O(V + E) O(V+E).

```
def dfs_recursivo(grafo, actual, visitados, orden):
    """DFS recursivo (usa call stack como pila)."""
    visitados.add(actual)
    orden.append(actual)
    for vecino in grafo.obtener_vecinos(actual):
        if vecino not in visitados:
            dfs_recursivo(grafo, vecino, visitados, orden)
```

4.5 Comparación BFS vs DFS y Aplicaciones Reales

Aspecto	BFS	DFS
Orden de exploración	Nivel por nivel	Profundidad primero

Aspecto	BFS	DFS
Estructura auxiliar	Cola (FIFO)	Pila (LIFO)
Camino más corto	Sí (en grafos no ponderados)	No
Detección de ciclos	Posible (con colores)	Excelente (backtracking)
Uso de memoria	Proporcional a anchura del grafo	Proporcional a profundidad
Aplicaciones típicas	Rutas más cortas, GPS, redes	Árboles de decisión, topología

Aplicaciones reales actuales (2026):

Redes sociales (Facebook, X, LinkedIn): recomendaciones de amigos mediante DFS/BFS en grafos de usuarios.

Sistemas de navegación GPS: BFS para rutas con menor número de intersecciones; Dijkstra (extensión ponderada) para distancias reales.

Internet y web crawling: los motores de búsqueda usan DFS para indexar páginas.

Bioinformática: ensamblaje de genomas y análisis de redes metabólicas.

Circuitos eléctricos y redes de distribución: detección de ciclos y caminos mínimos.

Inteligencia artificial: exploración de estados en juegos y planificación.

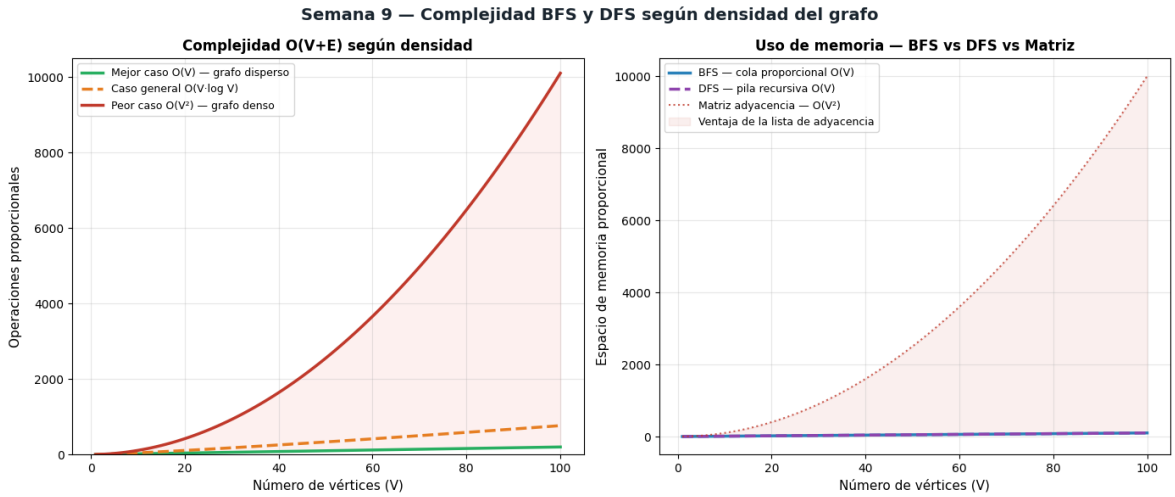
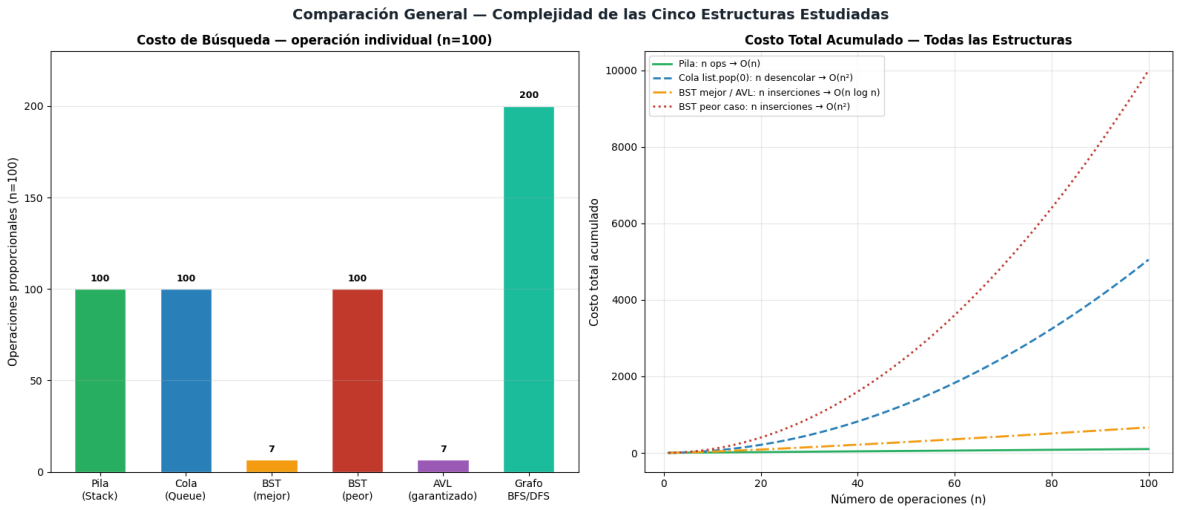


Tabla 1. Comparación Resumen

Estructura	Búsqueda prom.	Búsqueda peor	Inserción	Complejidad impl.	Caso de uso principal
Pila	O(n)	O(n)	O(1)	Baja	Historial, recursión, Undo/Redo
Cola	O(n)	O(n)	O(1)	Baja	Turnos, buffers, BFS
BST	O(log n)	O(n)	O(log n)	Media	Índices ordenados (riesgo degeneración)
AVL	O(log n)	O(log n)	O(log n)	Alta	Índices robustos, bases de datos
Grafo + BFS/DFS	O(V + E)	O(V + E)	O(1) por arista	Media-Alta	Redes, rutas, dependencias, IA

Grafica Comparativa de los Diferentes Métodos



Análisis Integrado: Importancia, Optimización y Perspectiva Globa

Se amplían las estrategias de optimización (elección de representación, balanceo como inversión, ordenamiento de vecinos, separación de responsabilidades) y las limitaciones reales (recursión en DFS, grafos no ponderados, implementación didáctica vs. producción). Se establecen conexiones explícitas: BFS usa Cola, DFS usa Pila, los árboles son grafos acíclicos.

Conclusión General

Se refuerzan las cinco conclusiones clave del documento original y se agregan reflexiones sobre el rol de estas estructuras en la inteligencia artificial, big data y sistemas distribuidos

de 2026. La maestría en estructuras de datos es la base del pensamiento algorítmico avanzado.

Referencias

Adelson-Velsky, G., & Landis, E. M. (1962). An algorithm for the organization of information. *Proceedings of the USSR Academy of Sciences, 146*, 263–266.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3.ª ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley.

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3.ª ed.). Addison-Wesley.

Python Software Foundation. (2024). *collections — Container datatypes*. https://docs.python.org/3/library/collections.html

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley.

Skiena, S. S. (2008). *The algorithm design manual* (2.ª ed.). Springer.